

Problem 3

3. [20p] Given an AVL designed using following data structure:

AVL: <root: AVLNode>

AVLNode: <int value, AVLNode left, AVLNode right, int height>

- [2p] Provide the representation invariant for this data structure to effectively represent an AVL.
- [3p] Provide the pseudocode of the iterative (non-recursive) versions of algorithms to insert, delete and search elements in the AVL.
- [6p] Prove that the previous algorithms preserve the representation invariant (it is true before and after the execution of each of the 3 methods).
- [7p] Implement the algorithms in Java, including the method boolean repOK() that checks the representation invariant, and provide meaningful tests, including test that checks repOK().

(a) Provide the representation invariant for this data structure to effectively represent an AVL.

AVL Representation:

- AVLNode: <int value, AVLNode left, AVLNode right, int height>
- AVL: <root: AVLNode>

AVL Representation Invariant:

$RI(T) = T$ is Acyclic w.r.t left and right

$\wedge \forall n \in \text{nodes}(T) : (\forall l \in \text{LeftSubtree}(n), l.\text{value} < n.\text{value}) \wedge (\forall r \in \text{RightSubtree}(n), r.\text{value} > n.\text{value})$

$\wedge \forall n \in \text{nodes}(T) : \text{balance}(n) \in \{-1, 0, 1\}$

$\wedge \forall n \in \text{nodes}(T) : n.\text{height} = \text{height}(n)$

where $\text{height}(n)$ is:

- If n is null: return 0
- Otherwise: return $1 + \max(\text{height}(n.\text{left}), \text{height}(n.\text{right}))$

$\text{balance}(n)$ is $\text{height}(n.\text{right}) - \text{height}(n.\text{left})$.

(b) Provide the pseudocode of the iterative (non-recursive) versions of algorithms to insert, delete and search elements in the AVL.

1. insert

```
1  AVLNode insert(root: AVLNode, val: int)
2      //record the visited node using stack
3      stack<AVLNode> stack = new stack()
4      AVLNode curr = root
5      AVLNode parent = null
6
7      if curr == null
8          return new AVLNode(val)
9
10     //go down
```

```

11     while curr != null
12         parent = curr
13         stack.push(parent)
14         if val < curr.value
15             curr = curr.left
16         else if val > curr.value
17             curr = curr.right
18         else
19             return root
20
21     // Create new node and link to parent
22     AVLNode newNode = new AVLNode(val)
23     if val < parent.value
24         parent.left = newNode
25     else
26         parent.right = newNode
27
28     // Pop the node to update height and rebalance
29     while !stack.isEmpty()
30         AVLNode node = stack.pop()
31         updateHeight(node)
32         AVLNode subTree = rebalance(node)
33
34     // If rotation happened, link the new subtree root to its parent
35     if !stack.isEmpty()
36         AVLNode ancestor = stack.readTop()
37         if ancestor.left == node
38             ancestor.left = subTree
39         else
40             ancestor.right = subTree
41     else
42         root = subTree
43
44     return root

```

2. search

```

1  AVLNode search(p: AVLNode, val: int)
2      while p != null && val != p.value
3          if val < p.value
4              p = p.left
5          else
6              p = p.right
7      return p

```

3. delete

```

1  AVLNode delete(root: AVLNode, val: int)
2      stack<AVLNode> stack = new stack()
3      AVLNode curr = root
4
5      // Find the node to delete and stored the visited node
6      while curr != null && curr.value != val
7          stack.push(curr)
8          if val < curr.value
9              curr = curr.left
10         else
11             curr = curr.right
12

```

```

13     // Not found
14     if curr == null
15         throw exception
16
17     // delete
18     // Case 1: node has two children
19     if curr.left != null && curr.right != null
20         //find successor
21         AVLNode successor = curr.right
22         stack.push(curr)
23
24         // Go left until null to find successor and let curr be successor
25         while successor.left != null
26             stack.push(successor)
27             successor = successor.left
28         curr.value = successor.value
29         curr = successor
30
31     // Case 2: Node has 0 or 1 child
32     AVLNode child = null
33     if curr.left != null
34         child = curr.left
35     else
36         child = curr.right
37
38     if stack.isEmpty()
39         return child
40     else
41         AVLNode parent = stack.readTop()
42         if parent.left == curr
43             parent.left = child
44         else:
45             parent.right = child
46
47     // pop the node to rebalance
48     while !stack.isEmpty()
49         AVLNode node = stack.pop()
50         updateHeight(node)
51         AVLNode subTree = rebalance(node)
52
53         if !stack.isEmpty()
54             AVLNode ancestor = stack.readTop()
55             if ancestor.left == node
56                 ancestor.left = subTree
57             else
58                 ancestor.right = subTree
59         else
60             root = subTree
61
62     return root

```

(c) Prove that the previous algorithms preserve the representation invariant (it is true before and after the execution of each of the 3 methods).

Search

```

1  while p != null && val != p.value
2      if val < p.value
3          p = p.left
4      else
5          p = p.right

```

It starts at the root and go down to find a value. It does not add nodes, remove nodes, or change any links between nodes. Since the algorithm never modifies T , the state of the tree after execution is same with the state before execution.

Insert

First, we record the visited node using stack. This part doesn't modify any node and doesn't break the structure.

Part 1

```

1  while curr != null
2      //...store the nodes
3      if val < curr.value {curr = curr.left}
4      else if val > curr.value {curr = curr.right}
5      ...
6      if val < parent.value {parent.left = newNode}
7      else {parent.right = newNode}

```

In this part, we go down the tree to find an empty position. If the value we want to add is smaller than the curr node, go left. If bigger, go right. Then We link the newNode to this position.

Since "smaller go left, bigger go right," we put `newNode` in the exact position required by the BST Invariant (i.e. $\forall l \in Left, l < n < r$).

We link a new leaf node to an existing tree. Since a leaf cannot link back to ancestors, the tree remains Acyclic.

Part 2

```

1  while !stack.isEmpty():
2      AVLNode node = stack.pop()
3      updateHeight(node)
4      AVLNode subTree = rebalance(node)
5      // ...link subTree to parent ...

```

We go back up the tree from the new node to the root using stack. For every node we going through, we recalculate its height. Then we call rebalance, thus we check if the left and right loose balance, then we rotate the nodes to fix it if needed

The `updateHeight(node)` method recalculate the height based on children ($1 + \max(h_L, h_R)$). Since we do this from the bottom up, every node's height is updated

The `rebalance(node)` method checks the balance factor ($h_R - h_L$).

If the factor is $\in \{-1, 0, 1\}$, do nothing. If the factor is ± 2 , rotations are needed

These rotations only move whole subtrees, so the BST order is preserved while we restore the balance

Thus the RI perserved

Delete

First, we find the node to delete and stored the visited node. This part doesn't modify any node and doesn't break the structure.

Part 1

```
1 // Case 1: Node has two children
2 if curr.left != null && curr.right != null:
3     // ... find successor
4     curr.value = successor.value
5     curr = successor // let curr be successor
6     // Case 2: Node has 0 or 1 child
7     // ...link parent to child ...
```

- If it has two children, we find the successor, let curr be successor.
- If it has 0 or 1 child, we just remove it and connect its parent to its child.

Replacing a node by a successor preserves the order. Because the successor is larger than every nodes in the left subtree and smaller than every node in the right subtree.

Connect a parent to a child by skipping the deleted node which also preserves the order of the remaining nodes

And the rotations in `rebalance` rearrange whole subtrees, so they also keep this BST order

Since we only remove link or pop stack instead of creating links to ancestors, thus it is acyclic

Part 2

```
1 while !stack.isEmpty():
2     AVLNode node = stack.pop()
3     updateHeight(node)
4     AVLNode subTree = rebalance(node)
5     // ...link subTree to parent
```

Similar to insert, removing a node might make one side be short. Thus we need to go back up the tree using stack and fix the heights and use rebalance to rotate the tree if node becomes unbalanced.

Deleting changes the height of the subtrees. But `stack.pop()` ensures we visit every ancestor of the deleted node. `updateHeight` ensures *n.height* matches the new tree.

Deleting can make a node's balance factor to become ± 2 . But the `rebalance` function can fix this by rotations

The `while` loop ensures we check every ancestor's height up to the root by rebalance

Thus after the loop finished, the tree is sorted and the height is balanced

Thus the invariant is preserved